Design Specification

I2C Master Controller with APB4 Interface

1. Document Information

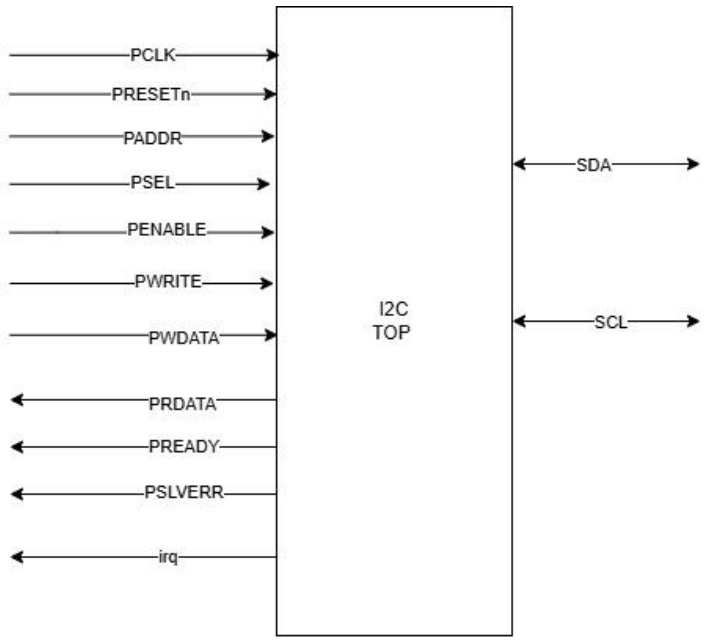
Field	Detail
Project Name	I2C Master Controller with APB4 Configuration Interface, part of the I2C Multi-Master / Slave Controller
Project Engineer	Bhavani Shri S
Version	v1.1
Date	October 4, 2026
Reference Specification	NXP/Philips I2C-Bus Specification and User Manual, UM10204

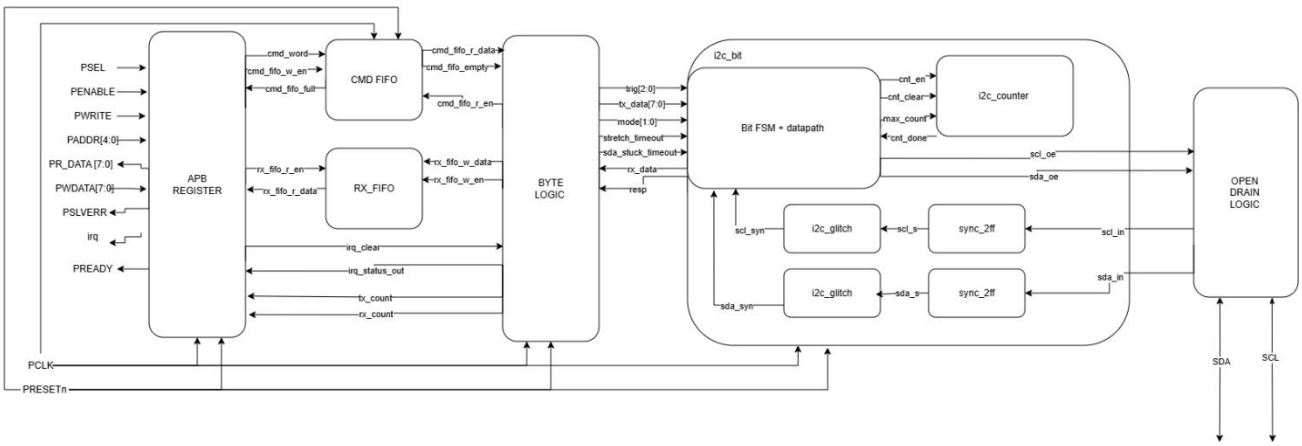
2. Overview

APB-controlled I²C multi-master/slave controller that bridges a processor's APB interface to an I²C bus. The design uses a command FIFO and byte-level FSM to manage I²C transactions, while a lower-level bit FSM handles SDA/SCL timing, ACK/NACK, arbitration, clock stretching and bus recovery. Synchronizers and programmable glitch filtering are used for reliable sampling of the external I²C lines, and an RX FIFO buffers received data back to the APB side

The design supports Standard (100 kHz), Fast (400 kHz) and Fast-mode Plus (1 MHz) speed grades, 7-bit and 10-bit addressing, master-side clock stretching, arbitration-loss detection, and a 9-clock bus-recovery sequence for a stuck SDA line. The current design does not expose a visible DONE or BUSY register. Software can monitor the TX/RX byte counters to track data progress and use IRQ_STATUS to detect error conditions. The actual transaction-completion state is internal to the byte FSM, so software does not have a dedicated indication for successful transaction completion.

2.1 Top-level architecture





Top-level block diagram of i2c_top with its submodules and the main signals between them.

Main signals between blocks.

From -> To	Signals	Meaning
APB master -> register	PADDR[4:0], PSEL, PENABLE, PWRITE, PWDATA[7:0]	APB4 transfer. The decode acts on the ACCESS phase (PSEL & PENABLE).
Register -> Command FIFO	cmd_word[11:0], cmd_fifo_w_en	Command word {PADDR[3:0], PWDATA} pushed for writes to offsets 0x00-0x0A when the FIFO is not full.
Command FIFO -> i2c_byte_fsm	cmd_in[11:0], cmd_fifo_empty	First-word-fall-through head of the FIFO; valid while cmd_fifo_empty = 0.
i2c_byte_fsm -> Command FIFO	cmd_fifo_r_en	Pop strobe. At most one command is consumed per PCLK cycle.
i2c_byte_fsm -> RX FIFO	rx_data[7:0], rx_fifo_w_en	One-cycle write pulse for each received byte.
Register <-> RX FIFO	rx_fifo_r_en, rx_fifo_r_data[7:0]	Pop on an APB read of offset 0x10; the head byte is presented combinationally (0x00 when empty).
Register -> i2c_byte_fsm	irq_clear	One-cycle pulse on a write to IRQ_CLEAR (0x0B).
i2c_byte_fsm -> Register	irq_status_out[5:0], tx_count[15:0],	Sticky error bits [5:0] and the byte counters read back

From -> To	Signals	Meaning
	rx_count[15:0]	at 0x11-0x15.
i2c_byte_fsm -> i2c_bit	trig[2:0], tx_data[7:0], mode[1:0], stretch_timeout[23:0], sda_stuck_timeout[23:0]	Command trigger and its operand, speed mode and the two timeouts.
i2c_bit -> i2c_byte_fsm	resp[2:0], rx_data[7:0]	Command status (INPROGRESS, then the result) and the received byte or sampled ACK bit.
i2c_bit <-> i2c_counter	en, clear, max_count -> done	Shared timing counter; done = (count >= max_count).
Pins -> sync_2ff -> i2c_glitch -> bit FSM	scl_in, sda_in -> scl_s, sda_s -> scl_syn, sda_syn	Input conditioning of the two asynchronous bus inputs.
i2c_bit -> open_drain	scl_oe, sda_oe	1 = drive the line low, 0 = release to the external pull-up.

3. Interfaces

Top-level module: i2c_top. It instantiates the APB4 register/decode logic, the command FIFO and RX FIFO, the byte-level sequencer, and, inside that, the bit-level engine, which instantiates sync_2ff, i2c_glitch and i2c_counter.

Signal Name	Direction	Width	Description
PCLK	Input	1	APB4 clock. (Single functional clock in the i2c blocks)
PRESETn	Input	1	APB4 active-low reset
PADDR [4:0]	Input	5	APB4 register address; selects the register or command word per Section 5.
PSEL	Input	1	Selects the peripheral, APB4 select for this slave.
PENABLE	Input	1	APB4 enable; APB access phase
PWRITE	Input	1	1 = write, 0 = read.
PWDATA [7:0]	Input	8	APB4 write data. Data that is being written
PRDATA [7:0]	Output	8	APB4 read data. Data that is returned during an APB read
PREADY	Output	1	Peripheral ready
PSLVERR	Output	1	APB error
irq	Output	1	Level interrupt: high whenever any bit of IRQ_STATUS [7:0] is set (combinational OR of the sticky bits, no mask register). Cleared by writing IRQ_CLEAR.
scl	Inout	1	Open-drain SCL pin to the external I2C bus. Driven low by the block or released to the external pull-up.
sda	Inout	1	Open-drain SDA pin to the external I2C bus. Driven low by the block or released to the external pull-up.

Every port belongs to the single PCLK domain except scl and sda, which are asynchronous, externally driven bus pins.

4. Clocking and Reset

Clocking. PCLK is the single functional clock for the entire block. It clocks the APB4 decode logic in i2c_top, both FIFOs, the byte-level FSM, the bit-level engine, the timing and timeout counters, the input synchronizers and the glitch filters. There is no separate, slower or gated clock.

Reset. PRESETn is active low and asserted asynchronously: every state and data register is captured on posedge PCLK or negedge PRESETn. De-assertion uses the same asynchronous sensitivity list, so a clean, glitch-free PRESETn is expected from outside the block.

Registers reset. All control, state and data registers are reset: FIFO read/write pointers and the overflow/underflow flags, the sticky illegal-access and command-FIFO-full bits, the byte-FSM state and

datapath, the configuration registers, irq_r, tx_cnt/rx_cnt, the command-runner handshake, the bit engine state and datapath, the two sync_2ff pairs, the two i2c_glitch filter/count registers and the i2c_counter count register.

Registers not reset. The two FIFO storage arrays (mem) are not reset. Their contents are never observable while the FIFO is empty: i2c_fifo forces r_data to 0x00 whenever the pointers are equal.

Clock-domain crossings. There are exactly two, both single-bit, external-asynchronous to PCLK:

- scl_in -> 2-flop synchronizer (sync_2ff) -> glitch filter (i2c_glitch) -> stretch/timeout logic and the bit-level FSM.
- sda_in -> 2-flop synchronizer (sync_2ff) -> glitch filter (i2c_glitch) -> arbitration comparison and the bit-level FSM.

Both synchronizers and both filters reset to logic 1, matching the pulled-up idle bus, so the conditioned bus reads idle immediately out of reset and no false START or busy condition is seen.

5. Register Map

All registers are byte wide and addressed by PADDR [4:0]. Writes to 0x00-0x0A are not stored in APB-side registers: i2c_top builds a 12-bit command word {PADDR [3:0], PWDATA [7:0]} and pushes it into the command FIFO. Configuration values therefore take effect when the command is popped by i2c_byte_fsm, in FIFO order. Reads from 0x10-0x15 return live status.

5.1 Register summary

Offset	Register Name	Width	Access	Reset Value	Description
0x00	I2C_SLV_ADDRH	2	W	0x0 (addrh = 2'b00)	Slave address bits [9:8] (10-bit addressing). Command header 0x0.
0x01	I2C_SLV_ADDRL	8	W	0x00	Slave address low byte: 7-bit address in [6:0], or bits [7:0] of a 10-bit address. Header 0x1.
0x02	MODE	3	W	0x0 (Standard, 7-bit)	[2:1] speed, [0] 10-bit addressing. Header 0x2.
0x03	SCL_TIMEOUT2	8	W	0x00	Clock-stretch timeout [23:16]. Header 0x3.
0x04	SCL_TIMEOUT1	8	W	0x00	Clock-stretch timeout [15:8]. Header 0x4.
0x05	SCL_TIMEOUT0	8	W	0x00	Clock-stretch timeout [7:0]. 24-bit value 0 disables the timeout. Header 0x5.
0x06	SDA_TIMEOUT2	8	W	0x00	SDA-stuck / bus-busy timeout [23:16]. Header 0x6.
0x07	SDA_TIMEOUT1	8	W	0x00	SDA-stuck / bus-busy timeout [15:8]. Header 0x7.
0x08	SDA_TIMEOUT0	8	W	0x00	SDA-stuck / bus-busy timeout [7:0]. 24-bit value 0 disables recovery; a busy bus then fails the START at once. Header 0x8.
0x09	TX_DATA	8	W	n/a	Queues a transmit byte. Header 0x9 (CMD_TX).
0x0A	RX_DATA	8	W	n/a	Queues a receive-byte request; PWDATA is ignored. Header 0xA (CMD_RX).
0x0B	IRQ_CLEAR	-	W	n/a	Any write pulses irq_clear for one PCLK cycle. Not queued; takes effect immediately.
0x0C-0x0F	RESERVED	-	-	n/a	Write: ignored, sets IRQ_STATUS[7]. Read: 0x00, sets IRQ_STATUS[7].
0x10	RX_FIFO	8	R	0x00 (FIFO empty)	Pops and returns the next received byte; reads 0x00 when the FIFO is empty.
0x11	TX_COUNT_HIGH	8	R	0x00	tx_count[15:8].
0x12	TX_COUNT_LOW	8	R	0x00	tx_count[7:0].
0x13	RX_COUNT_HIGH	8	R	0x00	rx_count[15:8].
0x14	RX_COUNT_LOW	8	R	0x00	rx_count[7:0].

Offset	Register Name	Width	Access	Reset Value	Description
0x15	IRQ_STATUS	8	R	0x00	Sticky status bits

The configuration registers are write-only: they cannot be read back, and a read of any offset outside 0x10-0x15 returns 0x00.

5.2 Bit fields and reset values

Register	Bits	Field	Access	Reset	Function
I2C_SLV_ADDRH	[7:2]	-	W	-	Ignored.
	[1:0]	ADDR_HI	W	2'b00	Address bits [9:8]. Used only when MODE[0] = 1; sent in the first 10-bit address byte {11110, ADDR_HI, R/W}.
I2C_SLV_ADDRL	[7:0]	ADDR_LO	W	0x00	7-bit mode: bits [6:0] are the slave address and bit 7 is ignored; the address byte is {ADDR_LO[6:0], R/W}. 10-bit mode: sent as the second address byte.
MODE	[7:3]	-	W	-	Ignored.
	[2:1]	SPEED	W	2'b00	00 Standard (100 kHz), 01 Fast (400 kHz), 10 Fast-mode Plus (1 MHz), 11 Standard.
	[0]	ADDR10	W	1'b0	0 = 7-bit addressing, 1 = 10-bit addressing.
SCL_TIMEOUT2/1/0	[7:0] each	STRETCH_TO	W	0x00 each	24-bit clock-stretch timeout in PCLK cycles, 0 = disabled. Maximum time SCL may stay low after the engine releases it (includes the 2 + FILTER_LEN cycle input latency).
SDA_TIMEOUT2/1/0	[7:0] each	SDA_STUCK_TO	W	0x00 each	24-bit timeout in PCLK cycles, 0 = disabled. Evaluated only when a START finds the bus not free (Section 6.7).
TX_DATA	[7:0]	TXD	W	-	Byte to transmit. Not stored in an APB register.
RX_DATA	[7:0]	-	W	-	Ignored; the write itself is the request.
IRQ_CLEAR	[7:0]	-	W	-	Ignored; any write clears all of IRQ_STATUS.
RX_FIFO	[7:0]	RXD	R	0x00	Received byte; 0x00 when the FIFO is empty.
TX_COUNT_HIGH/LOW	[7:0] each	tx_count	R	0x0000	16-bit count of data bytes whose ACK slot has completed (ACKed or NACKed). Address bytes are not counted. Wraps at 0xFFFF; not clearable except by reset.
RX_COUNT_HIGH/LOW	[7:0] each	rx_count	R	0x0000	16-bit count of received data bytes. Wraps at 0xFFFF; not clearable except by reset.
IRQ_STATUS	[0]	NACK	R	0	Slave NACKed an address byte or a transmitted data byte.
	[1]	ARB_LOST	R	0	Arbitration lost.
	[2]	SDA_ERR	R	0	SDA still low after the recovery sequence.
	[3]	SDA_REC	R	0	SDA was stuck low and the recovery sequence released it (informational).
	[4]	SCL_ERR	R	0	SCL held low past stretch_timeout, or SCL stuck low at START.
	[5]	BUS_BUSY	R	0	Bus not free when a START was requested and recovery is disabled.
	[6]	CMD_FIFO_FULL	R	0	A command write was dropped because the command FIFO was full.
	[7]	ILLEGAL	R	0	Illegal APB access .

All IRQ_STATUS bits are sticky. A new event wins over a clear presented in the same cycle.

5.3 Command word encoding

Every APB write to 0x00-0x0A becomes one 12-bit command word in the command FIFO. The header is the low four bits of the APB address, so the register offset and the command header are the same number.

12-bit command word format.

Bits	Field	Meaning
[11:8]	HDR	Command header = PADDR[3:0]. Selects the action (Table 3).
[7:0]	DATA	Command operand = PWDATA[7:0]. Meaning depends on HDR.

Command header values.

HDR	Mnemonic	APB offset	DATA bits used	Action when i2c_byte_fsm pops the word
0x0	CMD_ADDRH	0x00	[1:0]	addrh <= DATA[1:0].
0x1	CMD_ADDRL	0x01	[7:0]	addrl <= DATA.
0x2	CMD_MODE	0x02	[2:0]	mode_reg <= DATA[2:0] (speed and address size).
0x3	CMD_SCL_TIMEOUT_2	0x03	[7:0]	stretch_timeout[23:16] <= DATA.
0x4	CMD_SCL_TIMEOUT_1	0x04	[7:0]	stretch_timeout[15:8] <= DATA.
0x5	CMD_SCL_TIMEOUT_0	0x05	[7:0]	stretch_timeout[7:0] <= DATA.
0x6	CMD_SDA_TIMEOUT_2	0x06	[7:0]	sda_stuck_timeout[23:16] <= DATA.
0x7	CMD_SDA_TIMEOUT_1	0x07	[7:0]	sda_stuck_timeout[15:8] <= DATA.
0x8	CMD_SDA_TIMEOUT_0	0x08	[7:0]	sda_stuck_timeout[7:0] <= DATA.
0x9	CMD_TX	0x09	[7:0]	Transmit DATA. At the start of a transaction it also triggers START and the address phase; mid-transaction it sends the next byte (or forces a repeated START if the previous byte was a read).
0xA	CMD_RX	0x0A	none	Receive one byte. At the start of a transaction it triggers START and the address phase; consecutive CMD_RX words form a read burst.

Headers 0xB-0xE are never generated: offset 0x0B (IRQ_CLEAR) is handled outside the FIFO and writes above 0x0B are rejected. Header 0xF (CMD_IGNORE) is reserved; i2c_byte_fsm drops it if it is ever present, but no APB access produces it.

Configuration words (0x0-0x8) are consumed only while i2c_byte_fsm is in M_IDLE. A configuration word that reaches the head of the FIFO in the middle of a burst therefore ends the transaction with a STOP and is applied afterwards, so a configuration write can also be used to separate two transactions.

5.4 Access rules

Access	Offsets	Effect	Sets IRQ_STATUS[7]
Write	0x00-0x0A	Command word pushed into the command FIFO. If the FIFO is full the word is dropped and IRQ_STATUS[6] is set.	No
Write	0x0B	irq_clear pulse, immediate (not queued).	No
Write	0x0C-0x1F	Ignored.	Yes
Read	0x10-0x15	Returns the value in Section 5.1. A read of 0x10 pops the RX FIFO.	No
Read	0x00-0x0F, 0x16-0x1F	Returns 0x00.	Yes

PREADY is always 1 and PSLVERR is always 0, so illegal accesses complete normally and are visible only through IRQ_STATUS[7].

6. Functional Description

6.1 Micro-architecture

6.1.1 i2c_top (APB4 register / decode logic)

i2c_top contains no sequencing. It qualifies each APB transfer (apb_access = PSEL & PENABLE; PREADY is tied high, so this is exactly the one-cycle ACCESS phase), decodes the offset, forms the command word, drives the command FIFO write strobe, generates the irq_clear pulse, and holds two sticky status flops (command-FIFO-full and illegal-access) that follow the same policy as the sticky bits in i2c_byte_fsm. The read mux selects PRDATA combinationally from PADDR. irq is the OR of the eight status bits. The open-drain logic is implemented here(scl = scl_oe ? 1'b0 : 1'bz), similarly for sda; scl_in and sda_in read the pin, including the block's own drive.

6.1.2 i2c_fifo (command FIFO and RX FIFO)

A single-clock FIFO with parameters WIDTH and DEPTH (power of two, at least 2). The read and write pointers carry one extra bit, so full is count[ADDR_WIDTH] and empty is pointer equality. The FIFO is first-word-fall-through: r_data is the head entry, forced to 0x00 when empty. A write is accepted on w_en & `full` and a read on r_en & empty; a write when full or a read when empty is ignored. overflow and underflow are registered one-cycle flags that are not connected at i2c_top, so an RX FIFO overflow is not reported. The command FIFO is 12 bits wide, the RX FIFO 8 bits.

6.1.3 i2c_byte_fsm

i2c_byte_fsm turns the command stream into bus transactions. It has four parts:

- **Configuration registers**: addrh, addrl, mode_reg, stretch_timeout and sda_stuck_timeout. They are loaded only when a configuration word is popped in M_IDLE and feed i2c_bit directly (mode_reg[2:1] is the speed mode).
- **Command runner**: issues one i2c_bit command at a time. The Main FSM pulses go with a trigger code and operand; the runner registers trig and tx_data and sets busy, holds trig until i2c_bit shows resp = INPROGRESS (command accepted), releases trig to IDLE, then waits for resp to change, and finally pulses done with the result latched in res (status) and res_rx (rx_data of i2c_bit). res and res_rx remain valid until the next command completes.
- **Main FSM** (M_IDLE ... M_FLUSH): pops commands, builds address bytes, decides ACK or NACK for each received byte, chooses between continue, repeated START and STOP, and handles errors. A one-bit flag (iss) ensures each state issues its i2c_bit command exactly once.
- **Status**: the sticky irq_r[5:0] register, updated as irq_r <= (irq_r & ~irq_clear) | irq_set, and the 16-bit tx_cnt and rx_cnt counters. rx_fifo_w_en is a registered one-cycle pulse and rx_data is the byte it writes.

6.1.4 i2c_bit

i2c_bit executes one bus-level command (START or repeated START, TX byte, RX byte, STOP) at a time, generates the SCL/SDA timing and detects bus errors.

- **Input conditioning**: scl_in and sda_in pass through sync_2ff and i2c_glitch, giving scl_syn and sda_syn. bus_free = scl_syn & sda_syn.
- **Timing table**: the nanosecond constants for the selected speed mode are converted to PCLK cycles and rounded up, so no interval is shorter than the specified minimum.
- **Timing counter**: one shared i2c_counter. It is cleared on every state change, and max_count is chosen per state as (interval - 1), so each state lasts exactly the interval in cycles. In the four states that wait for SCL to rise (S_ST_SCLH, S_BIT_HIGH, S_SP_SCLH, S_REC_HIGH) the counter advances, and done is qualified, only while scl_syn = 1. The interval is therefore measured from the moment SCL is observed high.
- **Timeout counters**: stretch_cnt counts consecutive cycles spent in a wait-for-SCL-high state with scl_syn = 0 and raises stretch_to when it reaches stretch_timeout (0 disables). sda_stuck_cnt counts cycles in S_CHK while the bus is not free and raises sda_stuck_to when it reaches sda_stuck_timeout.

- **FSM and datapath** (S_IDLE ... S_ABORT): tx_buf and rx_buf shift registers, bit_idx, ack_phase, op_rx, ack_lat (the ACK/NACK bit to send after an RX byte), bus_owned (a START has been issued and no STOP yet), rec_mode and rec_cnt (bus recovery) and err_code (result reported from S_ABORT).
- **Line drive**: scl_oe and sda_oe are registers computed from the current state, so each line change appears one PCLK cycle after the state is entered. The engine never drives a line high.

6.1.5 sync_2ff, i2c_glitch and i2c_counter

- **sync_2ff**: two flops in series with asynchronous reset to RESET_VAL (1). Output is the second flop.
- **i2c_glitch**: holds a filtered value filt and counts consecutive samples that disagree with it; filt changes only after FILTER_LEN consecutive disagreeing samples, so shorter pulses are rejected. With en = 0 the module is a combinational pass-through (dout = din).
- **i2c_counter**: count increments while en is high and done is low, clears on clear, and done = (count >= max_count). The count output is not used by i2c_bit.

6.1.6 How the submodules interact

The following sequence describes a write of two bytes to a 7-bit slave (software writes I2C_SLV_ADDRL, TX_DATA, TX_DATA):

1. The three APB writes are decoded in i2c_top and pushed into the command FIFO as three 12-bit words.
2. i2c_byte_fsm (M_IDLE) pops the ADDRL word and applies it, then pops the first TX word, stores its data and moves to M_START.
3. M_START pulses go with TRIG_START. The command runner presents the trigger to i2c_bit, which performs the START and reports resp = IDLE; the runner pulses done.
4. M_ADDR1 issues TRIG_TX with {ADDRL[6:0], 0}. i2c_bit shifts the byte out MSB first, reads the slave's ACK bit and returns it in rx_data[0]. An ACK leads to M_DATA.
5. M_DATA issues TRIG_TX for the first data byte; on ACK tx_count increments and M_NEXT examines the FIFO head. The second TX word is popped and M_DATA sends it.
6. After the second byte M_NEXT finds no data command at the FIFO head, so M_STOP issues TRIG_STOP; when i2c_bit reports resp = IDLE the byte FSM returns to M_IDLE.

A read is the same except that M_DATA issues TRIG_RX with the ACK/NACK bit chosen from the FIFO head, and the byte returned in rx_data is pushed into the RX FIFO and counted in rx_count. Errors from i2c_bit return as a non-zero resp and are handled in M_ERR .

6.2 APB4 register access

i2c_top is a standard two-phase APB4 slave: a SETUP cycle (PSEL high, PENABLE low) followed by an ACCESS cycle (PSEL and PENABLE high). PREADY is tied high and PSLVERR tied low, so every access takes exactly one ACCESS cycle.

- **Write to 0x00-0x0A**: the command word is pushed into the command FIFO at the end of the ACCESS cycle if the FIFO is not full. If it is full the word is dropped and IRQ_STATUS[6] is set.
- **Write to 0x0B**: irq_clear is pulsed for one cycle regardless of PWDATA.
- **Write to 0x0C-0x1F**: no register effect; IRQ_STATUS[7] is set.
- **Read of 0x10**: returns the RX FIFO head and pops it at the end of the ACCESS cycle. A read while empty returns 0x00 and changes nothing.
- **Read of 0x11-0x15**: returns the corresponding live status byte.
- **Read of any other offset**: returns 0x00 and sets IRQ_STATUS[7].

6.3 Transaction model

There is no explicit START or STOP command. A run of consecutive CMD_TX / CMD_RX words in the command FIFO forms one bus transaction. The first data command that reaches the head of the FIFO while the engine is idle causes a START and an address phase. While the direction stays the same (TX after TX, or RX after RX) bytes are sent back to back. A change of direction (TX followed by RX or RX followed by TX) produces a repeated START and a new address phase. After each data byte, i2c_byte_fsm looks at the FIFO

head: if it is not a data command, because the FIFO is empty or the next word is a configuration word, a STOP is generated and the engine returns to idle.

The decision is taken once per byte, in the cycle after the byte completes. Commands that are already in the FIFO at that moment belong to the current transaction; commands written later start a new one.

6.4 START, addressing and repeated START

On the first data command of a transaction the engine issues a START (or, mid-transaction, a repeated START), then transmits the address byte(s):

- **7-bit addressing** (MODE[0] = 0): one address byte, {addrl[6:0], R/W}.
- **10-bit write**: two bytes, {11110, addrh[1:0], 0} then addrl[7:0].
- **10-bit read that starts a transaction**: the write-form sequence {11110, addrh[1:0], 0}, addrl[7:0] is sent first; the engine then issues a repeated START and sends {11110, addrh[1:0], 1}, after which data is received. This is the 10-bit read protocol of UM10204.
- **10-bit read reached by a direction change from a write**: the repeated START is followed by only the first byte, {11110, addrh[1:0], 1}.
- **Direction change from a 10-bit read to a write**: the repeated START is followed by {11110, addrh[1:0], 0} and addrl[7:0].

A NACK on an address byte sets IRQ_STATUS[0], stops the transaction with a STOP and discards the remaining queued data commands. START, repeated START and STOP waveforms are in Figures 4 to 6.

6.5 Data phase and ACK/NACK handling

TX byte. The engine drives the byte MSB first, then releases SDA and samples the slave's ACK bit. A NACK sets IRQ_STATUS[0] and ends the transaction with a STOP. The byte is still counted in tx_count.

RX byte. The engine releases SDA, receives eight bits MSB first, then drives the ACK/NACK bit itself. The bit is chosen when the command is issued to i2c_bit: ACK (0) if the command at the FIFO head at that moment is also a CMD_RX, otherwise NACK (1). A read burst must therefore be queued at least one command ahead of the byte in flight. If a CMD_RX arrives after the preceding byte was already NACKed, the preceding byte is correctly NACKed and the new byte is fetched with a repeated START and a new address phase.

6.6 Transaction continuation

After each completed data byte (state M_NEXT) the engine checks the FIFO head: a TX in the write direction or an RX in the read direction (and the previous byte was not NACKed) continues the burst; a data command of the opposite direction causes a repeated START and a fresh address phase; anything else causes a STOP and a return to idle.

6.7 Error handling

Each error is reported as a sticky bit in IRQ_STATUS and ends the current transaction. In every error case, TX/RX commands still queued for the failed transaction are popped and discarded once the transaction has ended (M_FLUSH), so a following configuration word or the next transaction's first data command is not mistaken for a continuation.

Error conditions, reporting and byte-FSM action.

Condition	Result code	IRQ bit	Detected / handled	STOP issued by i2c_byte_fsm
Slave NACK on an address byte or on a TX data byte	(ACK bit = 1)	[0]	Byte FSM reads the sampled ACK bit; goes to M_STOP, then M_FLUSH.	Yes
Arbitration lost	ARB_LOST	[1]	i2c_bit: a transmitted 1 (address or data bit, not the ACK bit) reads back 0, or SDA is low when a repeated START needs it high. Lines are released and i2c_bit returns to IDLE.	No
SDA stuck,	SDA_ERR	[2]	SDA still low after the 9-clock recovery	No (recovery already

Condition	Result code	IRQ bit	Detected / handled	STOP issued by i2c_byte_fsm
unrecoverable			and its STOP.	ended with a STOP)
SDA stuck, recovered	SDA_RECOVERY	[3]	Recovery released SDA. The START is retried once. If the retry hits SDA_RECOVERY again the transaction is abandoned and only bit [3] is set.	No
SCL held low too long	SCL_ERR	[4]	SCL low longer than stretch_timeout after the engine released it, or SCL low at START after sda_stuck_timeout expired. Lines released.	No
Bus not free at START	BUS_BUSY	[5]	SDA or SCL low at START with sda_stuck_timeout = 0, or SDA low at the end of the START setup time when no START was previously issued. Lines released.	No

START on a busy bus. If the bus is not free when a START is requested: with sda_stuck_timeout = 0 the START fails at once with BUS_BUSY. With a non-zero timeout the engine waits; if the bus becomes free the START proceeds. If SDA stays low with SCL high for the timeout, the engine runs 9 SCL pulses with SDA released, then a STOP (bus recovery); SDA high afterwards gives SDA_RECOVERY and low gives SDA_ERR. If SCL is the line held low when the timeout expires, the result is SCL_ERR.

6.8 Bit-level timing generation

i2c_bit derives all intervals from a fixed table of UM10204 minimum times, converted to PCLK cycles with NS2C(ns) = ceil(CLK_FREQ_MHZ x ns / 1000), so no interval is shorter than the specification minimum.

Timing constants (ns) and PCLK cycles at 100 MHz (Standard / Fast / Fast-mode Plus).

Interval	Standard (ns)	Fast (ns)	Fast+ (ns)	Cycles at 100 MHz (Std / Fast / Fast+)
tSU;STA (START setup)	4700	600	260	470 / 60 / 26
tHD;STA (START hold)	4000	600	260	400 / 60 / 26
tLOW	6000	1500	600	600 / 150 / 60
tHIGH	4000	1000	400	400 / 100 / 40
tSU;STO (STOP setup)	4000	600	260	400 / 60 / 26
tBUF (bus free)	4700	1300	500	470 / 130 / 50
Recovery half-cycle	5000	1250	500	500 / 125 / 50
T_HD_DAT (SDA hold after SCL falls)	300	300	300	30

SCL high time is counted from the moment SCL is observed high after the synchronizer and filter, so tHIGH is never shorter than the target. This also provides clock-stretch handling and wired-AND clock synchronization with other masters. The resulting SCL frequency is slightly below nominal and never above it (about 99 kHz, 391 kHz and 943 kHz at 100 MHz); the shortfall is a fixed number of PCLK cycles of synchronizer and filter latency, so it grows at slower clocks (about 806 kHz in Fast-mode Plus at 25 MHz).

SDA data hold. After SCL is pulled low the engine keeps SDA unchanged for T_HD_DAT = 300 ns (rounded up to whole cycles) before moving SDA to the next bit value. The hold is carved out of tLOW, so the SCL low time is unchanged (Figure 8).

Per-state intervals. S_ST_REL: one cycle for an initial START, tLOW for a repeated START. S_ST_SCLH: tSU;STA from observed SCL high. S_ST_SDAL: tHD;STA. S_HOLD: at least T_HD_DAT before a START or STOP, none before TX/RX. S_BIT_SET: T_HD_DAT. S_BIT_LOW: tLOW - T_HD_DAT. S_BIT_HIGH: tHIGH from observed high. S_SP_SDAL: tLOW. S_SP_SCLH: tSU;STO from observed high. S_SP_SDAH: tBUF. Each recovery state: one half-cycle. Every state lasts at least one cycle.

Clock requirement. CLK_FREQ_MHZ must match the real clock. A clock of 20 MHz or more is recommended and 10 MHz is the absolute minimum for Fast-mode Plus, because the low phase must be longer than the synchronizer and filter latency.

6.9 Input conditioning

scl_in and sda_in each pass through a 2-flop synchronizer and a digital glitch filter. The filter output changes only after the input has disagreed with it for FILTER_LEN consecutive PCLK cycles, so pulses shorter than FILTER_LEN cycles are rejected. A change on the pin appears on scl_syn / sda_syn after about 2 + FILTER_LEN PCLK cycles (plus up to one cycle of sampling alignment). The filter can be bypassed per line at elaboration time (GLITCH_FILTER_EN = 0); it is not intended to be switched while the bus is active.

6.10 Transfer completion and status polling

The block has **no done, busy or FIFO-level register and no completion interrupt**. irq reports only the events in IRQ_STATUS: six bus events (five errors and the informational SDA-recovered bit), command-FIFO overflow and illegal access. Software determines progress and completion as follows.

What software can observe.

Question	Where to look	How it behaves
Has a TX data byte finished?	TX_COUNT (0x11/0x12)	Increments by 1 when the byte's ACK slot has completed, whether the slave ACKed or NACKed. Address bytes and configuration writes are not counted.
Has an RX byte arrived?	RX_COUNT (0x13/0x14), then RX_FIFO (0x10)	rx_count increments in the same cycle the RX FIFO write pulse is generated; the byte is in the FIFO one cycle later. Back-to-back APB accesses are at least two cycles apart, so a read of RX_FIFO that follows a read of RX_COUNT always sees the byte.
Did something fail?	irq, IRQ_STATUS (0x15)	Sticky error bits . A NACK is reported before the STOP has been sent.
Has the STOP completed and the bus gone free?	Not observable in registers	Wait a bounded time after the last count update (see below), or watch the bus.
Has the command FIFO drained?	Not observable in registers	Implied by the counters reaching the expected values.

Expected software sequence for a write of N bytes:

1. Optionally clear IRQ_STATUS (write 0x0B), then read TX_COUNT to get the starting value c0.
2. Queue the configuration words (if any) and N CMD_TX words. Queue the whole transaction promptly: if the FIFO runs empty between bytes, a STOP is issued (Section 6.3).
3. Poll until TX_COUNT - c0 (modulo 65536) equals N, or irq / IRQ_STATUS shows an error, with a software timeout in case neither happens (for example, an endless clock stretch with stretch_timeout = 0).
4. If the bus must be known free before the next action, wait for the STOP time below.

For a read of N bytes, queue N CMD_RX words, poll RX_COUNT in the same way, then read RX_FIFO N times. Because the RX FIFO returns 0x00 when empty, use RX_COUNT, not the value read, to know that data is present.

Error cases. An address NACK sets IRQ_STATUS[0] and tx_count does not advance. A data NACK sets IRQ_STATUS[0] and tx_count does advance for that byte, so software must check IRQ_STATUS as well as the count. For ARB_LOST, SDA_ERR, SCL_ERR and BUS_BUSY the counters stop where they were. After any error the commands already queued for the transaction are discarded without being counted.

STOP time. From the last byte's count update, the STOP takes about T_HD_DAT + tLOW + tSU;STO + tBUF plus a few PCLK cycles of handshake and synchronizer/filter latency: about 15 us in Standard mode, 3.7 us in Fast mode and 1.7 us in Fast-mode Plus.

Separating transactions. To force a STOP between two transactions, either wait for the first transaction's count before queuing the second, or place a configuration word (for example a rewrite of I2C_SLV_ADDRL) between them.

6.11 Interrupt behaviour and latency

irq is the combinational OR of the eight sticky IRQ_STATUS flops (no extra register stage, no mask). It is a level output: it stays high until the status bits are cleared by a write to IRQ_CLEAR. Bit [3] (SDA recovered) is informational and also raises irq.

Latency is given in PCLK cycles from the **event cycle**, the cycle in which the condition is first evaluated true by the logic named in the table, to the cycle in which irq is first seen high. A value of 1 means irq is high in the cycle immediately after the event cycle. The figures assume the logic itself is not delayed; events that originate on the bus add the input conditioning latency (2 + FILTER_LEN cycles), and SCL_ERR and BUS_BUSY add the programmed timeout.

irq assertion and de-assertion latency.

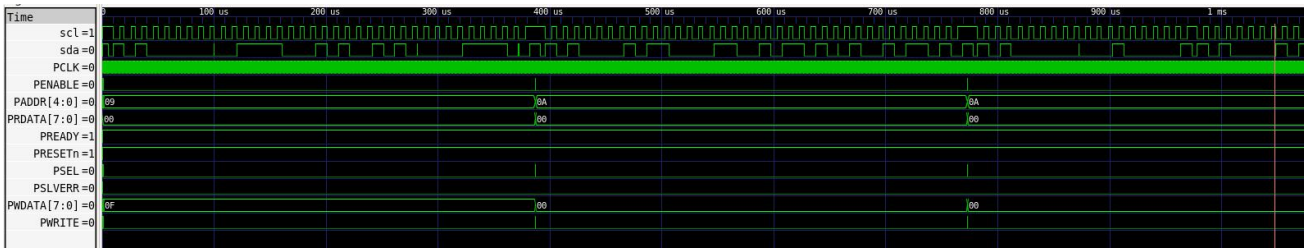
Source	Bit	Event cycle	Cycles to irq	Path
Slave NACK	[0]	i2c_bit samples the ACK bit at the end of the ACK-bit SCL high time.	3	bit engine -> runner -> byte FSM -> irq_r
Arbitration lost	[1]	i2c_bit detects the mismatch (or SDA low at a repeated START).	5	S_ABORT -> resp -> runner -> M_ERR -> irq_r
SDA error	[2]	End of the recovery STOP bus-free time with SDA still low.	4	resp -> runner -> M_ERR -> irq_r
SDA recovered	[3]	End of the recovery STOP bus-free time with SDA high.	3	resp -> runner -> byte FSM -> irq_r
SCL error	[4]	stretch_cnt reaches stretch_timeout (or SCL low at START after sda_stuck_timeout).	5	S_ABORT -> resp -> runner -> M_ERR -> irq_r
Bus busy	[5]	START finds the bus not free with sda_stuck_timeout = 0 (or SDA low at the end of START setup).	5	S_ABORT -> resp -> runner -> M_ERR -> irq_r
Command FIFO full	[6]	ACCESS cycle of the write that finds the FIFO full.	1	decode -> sticky flop
Illegal access	[7]	ACCESS cycle of the illegal read or write.	1	decode -> sticky flop
De-assertion (IRQ_CLEAR write)	all	ACCESS cycle of the write to 0x0B.	1 (irq low)	decode -> clears all sticky flops

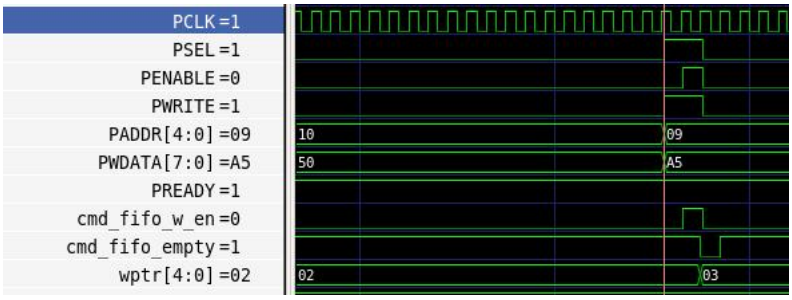
De-assertion. The irq_clear pulse is generated combinationally in the ACCESS cycle and clears every sticky bit at the edge that ends it, so irq is low in the first cycle after the IRQ_CLEAR write completes. A read of IRQ_STATUS immediately afterwards returns 0x00 unless a new event has occurred. A new event in the same cycle as the clear wins and keeps its bit set.

Ordering. IRQ_CLEAR is not queued behind pending commands. A NACK is reported before the STOP that follows it has been sent, so irq can rise while the bus is still active.

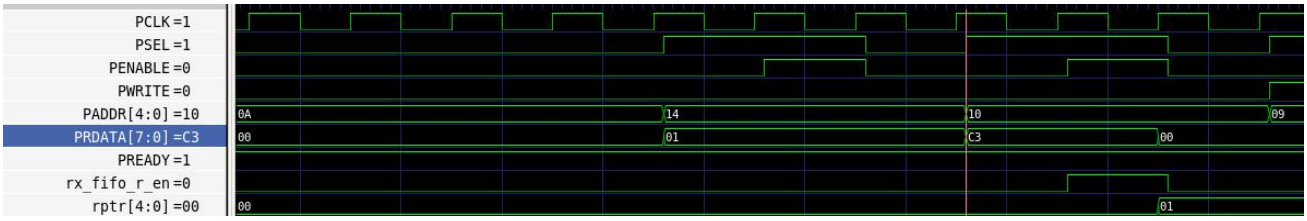
6.12 Bus and register timing diagrams

The APB diagrams are cycle accurate. The bus diagrams are not to scale: interval lengths depend on the speed mode (Section 6.8), and each scl_oe or sda_oe edge appears one PCLK cycle after its state is entered. SCL and SDA are shown as the resulting wired-AND line levels; rise times are not drawn.

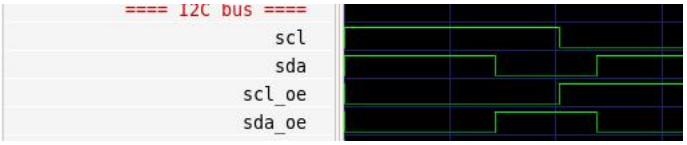




APB4 write of 0xA5 to TX_DATA (0x09).



APB4 read of RX_FIFO (0x10).



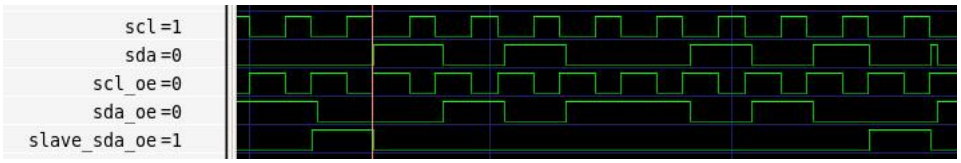
START on an idle bus.



Repeated START from S_HOLD, shown after a byte the master ACKed. SDA is released while SCL is low, SCL is released, then SDA falls while SCL is high.



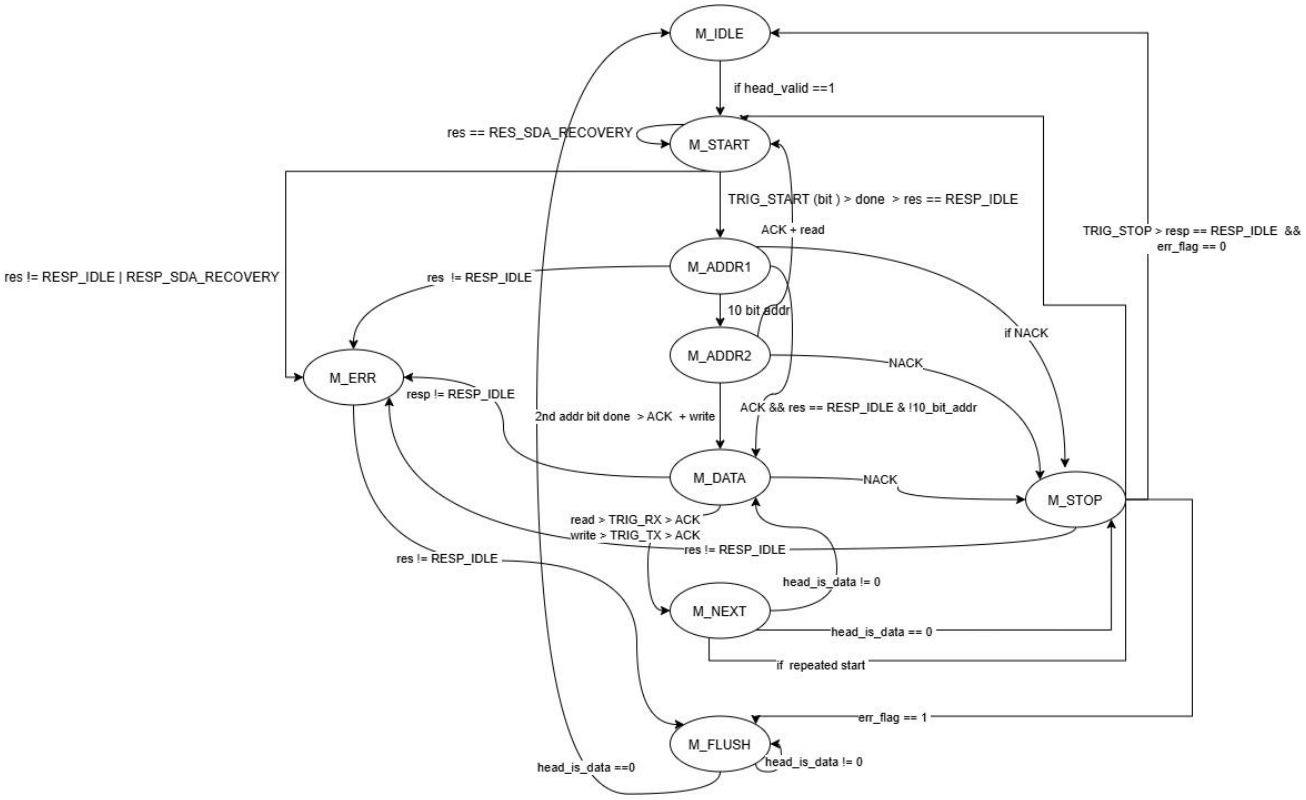
STOP signal genertion



One-byte write transfer with ACK: START, address byte 0xA5, slave ACK(slave_sda_oe) on the ninth clock.

7. FSM / State Transitions

7.1 Byte-level FSM (i2c_byte_fsm)

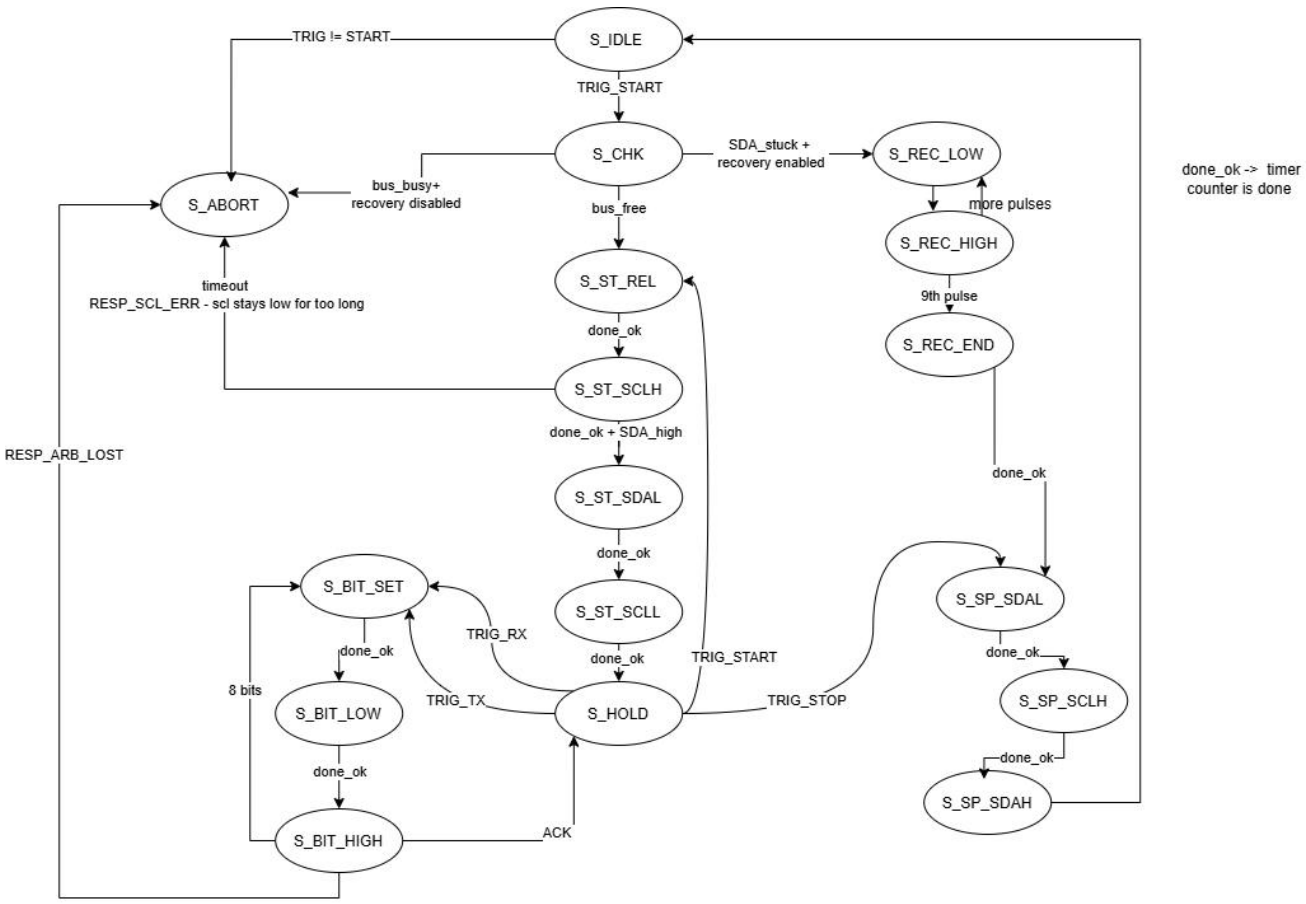


State transition diagram of i2c_byte_fsm.

Current state	Condition	Next state	What happens
M_IDLE	CMD_TX / CMD_RX	M_START	Start transaction
M_IDLE	Configuration command	M_IDLE	Update configuration
M_START	done & RESP_IDLE	M_ADDR1	START successful
M_START	SDA recovery + first retry	M_START	Retry START

Current state	Condition	Next state	What happens
M_START	SDA recovery + already retried	M_ERR	Error
M_START	Other error	M_ERR	Error
M_ADDR1	bit error	M_ERR	Error
M_ADDR1	address NACK	M_STOP	Set NACK + stop
M_ADDR1	address accepted, no second byte	M_DATA	Data phase
M_ADDR1	10-bit second byte required	M_ADDR2	Send second address byte
M_ADDR2	bit error	M_ERR	Error
M_ADDR2	NACK	M_STOP	Set NACK + stop
M_ADDR2	10-bit write	M_DATA	Data phase
M_ADDR2	10-bit read	M_START	Repeated START
M_DATA	bit error	M_ERR	Error
M_DATA	successful READ	M_NEXT	Push RX data
M_DATA	successful WRITE + ACK	M_NEXT	Increment TX count
M_DATA	WRITE + NACK	M_STOP	Set NACK + stop
M_NEXT	next TX same direction	M_DATA	Next data byte
M_NEXT	next RX same direction	M_DATA	Next data byte
M_NEXT	direction changes	M_START	Repeated START
M_NEXT	no data command	M_STOP	End transaction
M_STOP	STOP error	M_ERR	Error
M_STOP	success + err_flag=0	M_IDLE	Transaction complete
M_STOP	success + err_flag=1	M_FLUSH	Clear queued data
M_ERR	any handled error	M_FLUSH	Set IRQ + flush
M_FLUSH	data command remains	M_FLUSH	Pop command
M_FLUSH	no data command	M_IDLE	Return idle

7.2 Bit-level engine FSM (i2c_bit)



State transition diagram of i2c_bit.

Current state	Condition	Next state	What happens
S_IDLE	trig == TRIG_START	S_CHK	START command is accepted. resp becomes RESP_INPROGRESS; bus-free check begins.
S_IDLE	trig == TRIG_TX or TRIG_RX	S_ABORT	TX/RX is not allowed before START. err_code = RESP_BUS_BUSY.
S_IDLE	trig == TRIG_STOP	S_ABORT	STOP while already idle is treated as a no-op. err_code = RESP_IDLE.
S_IDLE	No valid trigger	S_IDLE	Waits for a command.
S_CHK	bus_free == 1	S_ST_REL	Bus is free; continue START generation.
S_CHK	Bus not free + sda_stuck_timeout == 0	S_ABORT	Immediately reports RESP_BUS_BUSY.
S_CHK	SDA stuck timeout reached + scl_syn == 1	S_REC_LOW	SDA is stuck low while SCL is high; starts 9-clock bus recovery.
S_CHK	SDA stuck timeout reached + scl_syn == 0	S_ABORT	SCL is also held low; reports RESP_SCL_ERR.

Current state	Condition	Next state	What happens
S_CHK	Bus not free and timeout not yet reached	S_CHK	Continues waiting/checking the bus.
S_ST_REL	done_ok	S_ST_SCLH	Releases SDA and waits for the START setup phase.
S_ST_REL	done_ok == 0	S_ST_REL	Continues waiting for the required timing.
S_ST_SCLH	stretch_to	S_ABORT	SCL remains low too long; reports RESP_SCL_ERR.
S_ST_SCLH	done_ok + sda_syn == 1	S_ST_SDAL	SCL is high and SDA is high; ready to generate START.
S_ST_SCLH	done_ok + sda_syn == 0 + bus_owned == 1	S_ABORT	Another master pulled SDA low; reports RESP_ARB_LOST.
S_ST_SCLH	done_ok + sda_syn == 0 + bus_owned == 0	S_ABORT	Bus is still busy; reports RESP_BUS_BUSY.
S_ST_SCLH	None of above	S_ST_SCLH	Continues waiting for SCL high/timing completion.
S_ST_SDAL	done_ok	S_ST_SCLL	Pulls SDA low while SCL is high, creating the START condition.
S_ST_SDAL	done_ok == 0	S_ST_SDAL	Continues START hold timing.
S_ST_SCLL	Always	S_HOLD	Pulls SCL low, marks bus_owned = 1, sets resp = RESP_IDLE; START is complete.
S_HOLD	trig == TRIG_START + done_ok	S_ST_REL	Generates a repeated START while the bus is already owned.
S_HOLD	trig == TRIG_TX	S_BIT_SET	Latches tx_data, sets TX mode, initializes bit_idx = 7; starts byte transmission.
S_HOLD	trig == TRIG_RX	S_BIT_SET	Clears RX buffer, sets RX mode, latches ACK/NACK value, initializes bit_idx = 7; starts byte reception.
S_HOLD	trig == TRIG_STOP + done_ok	S_SP_SDAL	Begins STOP sequence.
S_HOLD	No valid trigger	S_HOLD	Keeps the bus owned and waits for the next command.
S_BIT_SET	done_ok	S_BIT_LOW	Maintains SDA for T_HD_DAT before changing the SDA bit.
S_BIT_SET	done_ok == 0	S_BIT_SET	Continues SDA hold timing.
S_BIT_LOW	done_ok	S_BIT_HIGH	Drives the required SDA value and completes the SCL-low period.
S_BIT_LOW	done_ok == 0	S_BIT_LOW	Continues SCL-low timing.
S_BIT_HIGH	stretch_to	S_ABORT	SCL is held low too long by the bus; reports RESP_SCL_ERR.

Current state	Condition	Next state	What happens
S_BIT_HIGH	done_ok + TX + sent 1 but sda_syn == 0	S_ABORT	Arbitration is lost because another master forced SDA low. Reports RESP_ARB_LOST.
S_BIT_HIGH	done_ok + normal data bit + bit_idx != 0	S_BIT_SET	Samples/handles the current bit, decrements bit_idx, and prepares the next bit.
S_BIT_HIGH	done_ok + last data bit (bit_idx == 0)	S_BIT_SET	Completes 8 data bits and sets ack_phase = 1 to generate the 9th ACK bit.
S_BIT_HIGH	done_ok + ack_phase == 1	S_HOLD	ACK/NACK bit is complete. Stores rx_data, sets resp = RESP_IDLE, and waits for the next command.
S_SP_SDAL	done_ok	S_SP_SCLH	Holds SDA low while SCL is low, preparing for STOP.
S_SP_SCLH	stretch_to	S_ABORT	SCL remains low too long; reports RESP_SCL_ERR.
S_SP_SCLH	done_ok	S_SP_SDAH	Releases SCL and waits for it to become high.
S_SP_SDAH	done_ok	S_IDLE	Releases SDA while SCL is high, creating STOP. Clears bus_owned; normal completion gives RESP_IDLE.
S_REC_LOW	done_ok	S_REC_HIGH	Drives SCL low for the recovery clock's low half-cycle.
S_REC_HIGH	stretch_to	S_ABORT	SCL is held low too long during recovery; reports RESP_SCL_ERR.
S_REC_HIGH	done_ok + rec_cnt == 8	S_REC_END	Ninth recovery clock is complete.
S_REC_HIGH	done_ok + rec_cnt != 8	S_REC_LOW	Increments recovery counter and generates another recovery clock.
S_REC_END	done_ok	S_SP_SDAL	Recovery clocks are finished; begins normal STOP sequence with rec_mode = 1.
S_ABORT	Always	S_IDLE	Releases bus ownership, clears recovery mode, puts the error code into resp, and returns to idle.
Any undefined state	Any	S_IDLE	Default safety transition back to S_IDLE.

7.3 Command handshake (trig / resp)

i2c_byte_fsm and i2c_bit communicate through a level-trigger handshake. trig is accepted only while i2c_bit is in S_IDLE (bus free) or S_HOLD (bus owned); it is ignored in any other state. When the command is accepted resp becomes INPROGRESS; the caller then returns trig to IDLE. When the command finishes, resp holds the result until the next command is accepted.

trig encodings.

Name	Value	Meaning
TRIG_IDLE	0	No command.
TRIG_START	1	From S_IDLE: START. From S_HOLD: repeated START (after at least T_HD_DAT of SCL low). Leaves SCL low and the engine in S_HOLD.

Name	Value	Meaning
TRIG_TX	2	Send tx_data[7:0] MSB first, then read the slave's ACK bit into rx_data[0] (0 = ACK, 1 = NACK). Valid only in S_HOLD.
TRIG_RX	3	Receive eight bits, then drive the ACK bit tx_data[0] (0 = ACK, 1 = NACK), latched at acceptance. rx_data[7:0] = received byte.
TRIG_STOP	4	Generate STOP and wait the bus-free time. In S_IDLE it is a no-op returning RESP_IDLE.

resp encodings.

Name	Value	Meaning
RESP_IDLE	0	Success.
RESP_INPROGRESS	1	Command running.
RESP_ARB_LOST	2	A transmitted 1 read back as 0 (data bits only), or SDA low when a START or repeated START needed it high.
RESP_SDA_ERR	3	SDA stuck low and still low after bus recovery.
RESP_SDA_RECOVERY	4	SDA was stuck low; 9-clock recovery plus STOP released it.
RESP_SCL_ERR	5	SCL held low longer than stretch_timeout, or stuck low at START.
RESP_BUS_BUSY	6	Bus not free when a START was requested, or TX/RX issued with no START.

Handshake latency: i2c_bit moves to resp = INPROGRESS one cycle after it samples trig; trig is held for two cycles. After the result appears on resp the runner pulses done one cycle later, and the byte FSM acts on done in the following cycle.

8. Reference Specification and Supported Features

Reference Specification: NXP/Philips I2C-Bus Specification and User Manual, UM10204.

Feature (from reference spec)	Spec Section/Clause	Supported? (Yes / Partial / No)	Notes
START / repeated START generation	3.1.4, 3.1.10	Yes	-
STOP generation	3.1.5	Yes	-
7-bit addressing	3.1.11	Yes	-
10-bit addressing	3.1.11	Yes	Write, read and direction-change forms; see Section 6.4.
Standard-mode (100 kHz)	5, Table 10	Yes	-
Fast-mode (400 kHz)	5, Table 10	Yes	-
Fast-mode Plus (1 MHz)	5, Table 10	Yes	-
Master-transmitter: receive ACK/NACK	-	Yes	NACK reported in IRQ_STATUS[0].
Master-receiver: generate ACK/NACK, NACK on last byte	-	Yes	ACK only if another CMD_RX is already queued (Section 6.5).
SDA hold time after SCL falls (internal hold)	-	Yes	T_HD_DAT = 300 ns.
Clock stretching (master waits for a stretched SCL)	3.1.9	Yes	Programmable timeout; a timeout is flagged (IRQ_STATUS[4]), not auto-recovered.
Clock synchronization (wired-AND SCL)	-	Yes	SCL high time is counted from observed SCL high.
Multi-master arbitration-loss detection	3.1.6	Yes	A transmitted 1 read back as 0, or SDA low at a (repeated) START.
Arbitration recovery / retry	3.1.6	No	Loss is reported and the transfer abandoned; no

Feature (from reference spec)	Spec Section/Clause	Supported? (Yes / Partial / No)	Notes
			automatic retry.
Bus clear (stuck SDA)	3.1.16	Yes	9 clock pulses plus STOP; one retry of the initiating START.
Input spike suppression (tSP, 50 ns for Fast and Fast+)	-	Partial	Digital filter rejects pulses shorter than FILTER_LEN PCLK cycles. Meeting 50 ns needs FILTER_LEN x PCLK period of at least 50 ns (for example FILTER_LEN >= 5 at 100 MHz).
General call address (0x00)	3.1.13	No	Not implemented.
Software reset via general call	3.1.13	No	Not implemented.
Reserved addresses (START byte, CBUS, device ID, Hs master code)	-	No	No special handling; the address registers accept any value.
High-speed mode (3.4 MHz) and Ultra Fast-mode	-	No	Out of scope.
Slave mode (address match, ACK generation, slave stretching)	3.1	No	Out of scope for this module; see Section 10.
SMBus timeout / PEC	n/a (SMBus extension)	No	Not part of this project's scope.

9. Parameters

Top-level parameters (i2c_top)

Parameter	Legal Range	Default	Effect
CLK_FREQ_MHZ	>= 10 (>= 20 recommended)	100	PCLK frequency in MHz; all timing constants derive from it. Below about 10 MHz Fast-mode Plus margins collapse.
FILTER_LEN	>= 1	3	Glitch-filter length in PCLK cycles; shorter pulses are rejected.
GLITCH_FILTER_EN	0 or 1	1	1 = filter active, 0 = bypass (combinational pass-through).
CMD_FIFO_DEPTH	power of 2, >= 2	1024	Command FIFO entries (12 bits each); checked at elaboration.
RX_FIFO_DEPTH	power of 2, >= 2	1024	RX FIFO entries (8 bits each); checked at elaboration.

Internal parameters and derived values

Parameter	Legal Range	Default	Effect
CMD_HEADER_WIDTH (i2c_byte_fsm)	4 only	4	Width of the command-word header; checked at elaboration.
DATA_WIDTH (i2c_byte_fsm)	8 only	8	Width of command data and of TX/RX data; checked at elaboration.
WIDTH (i2c_bit)	>= CNT_WIDTH	24	Width of stretch_timeout and sda_stuck_timeout (3 x 8 to match the timeout command words).
CNT_WIDTH (i2c_bit, derived)	derived	10 at 100 MHz	$clog2(NS2C(6000) + 1), sized from the largest constant (Standard-mode tLOW).
RESET_VAL (sync_2ff, i2c_glitch)	1'b0 or 1'b1	1'b1	Reset value of the synchronizer and filter; 1 matches the pulled-up idle bus.
WIDTH, DEPTH (i2c_fifo)	DEPTH power of 2, >= 2	12/8, 1024	Entry width and depth; set by the top-level parameters above.

10. Known Limitations

- Master mode only. Slave mode (address match, ACK generation, slave clock stretching, general call) is out of scope for this controller.

- No done, busy or FIFO-level status and no completion interrupt. Completion is inferred from TX_COUNT / RX_COUNT and IRQ_STATUS, and the end of the STOP condition is not observable in registers. The counters are 16 bits, wrap, and cannot be cleared except by reset.
- irq has no mask or enable register and includes the informational SDA-recovered bit.
- A transaction ends when the command FIFO has no data command at the head after a byte. If software queues bytes more slowly than the bus consumes them, the transaction is split by a STOP. A configuration word at the FIFO head also ends a transaction.
- An RX byte is ACKed only if the next RX_DATA write is already queued when the byte's ACK/NACK is committed. A request that arrives one byte late completes correctly but through a repeated START and a new address phase.
- Command FIFO full: the write is dropped and only IRQ_STATUS[6] is set. A dropped command in the middle of a transaction changes what is sent, and the FIFO level cannot be read.
- RX FIFO overflow and underflow are not reported. A byte received while the RX FIFO is full is dropped but still counted in rx_count, and an empty-FIFO read returns 0x00, which is also a valid data value.
- After an error, only the data commands already queued for the failed transaction are discarded. Commands written afterwards start a new transaction.
- Multi-master arbitration loss is detected and the transfer abandoned, but there is no automatic retry or back-off; software observes IRQ_STATUS[1] and re-issues the transfer.
- A clock-stretch timeout is flagged but has no recovery action beyond ending the transaction. With stretch_timeout = 0 an endless stretch hangs the engine.
- If the START retry after SDA recovery hits a second recovery, the transaction is abandoned and only IRQ_STATUS[3] is set.
- Configuration registers are write-only and take effect only when popped while the engine is idle, in FIFO order.
- The SCL frequency is slightly below nominal because of synchronizer and filter latency; the shortfall grows at lower PCLK frequencies.
- DATA_WIDTH is a parameter of the byte-level FSM but the design is correct only for DATA_WIDTH = 8; other values are rejected by an elaboration-time check.
- General-call recognition and the software-reset broadcast are not implemented. No SMBus timeout or packet-error checking.
- PRESETn de-assertion is not internally re-synchronized; a clean, glitch-free reset is assumed to be supplied from outside the block.
- Illegal APB accesses complete normally (PSLVERR is never asserted) and are reported only through IRQ_STATUS[7].